

Upwind Home Assignment Submission

Candidate: Bar carmeli

Email: barbar0608@gmail.com

Phone: 0542330666

GitHub Repository:

<https://github.com/ShadowNightCoder/home-assignment>

Executive Summary

I have successfully implemented all three tasks, including the UI bonus requirements for Tasks 1, 2, and 3.

All source code, setup instructions, and detailed technical documentation are available in the linked GitHub repository above.

Below is a overview and visual proof of the implemented solutions.

Task 1: Email Phishing Detector

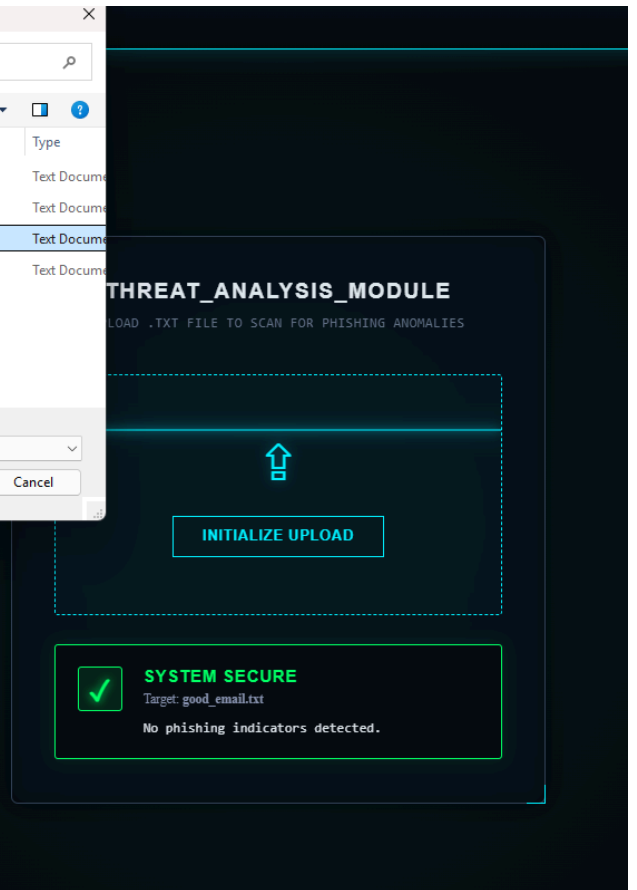
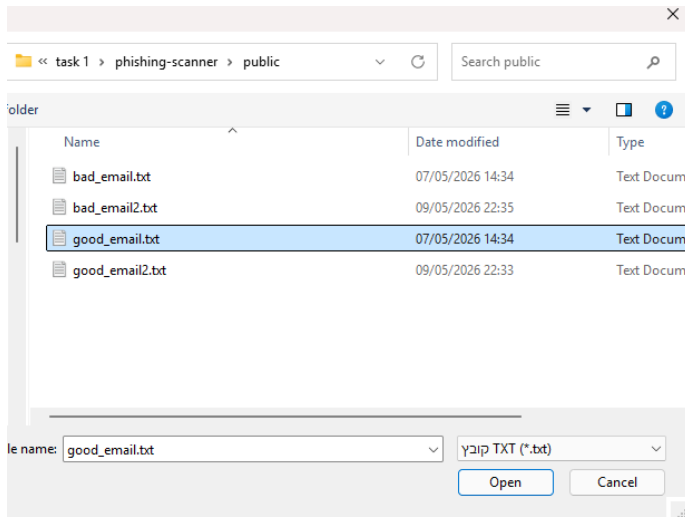
Objective: Develop a tool to scan email content for phishing indicators.

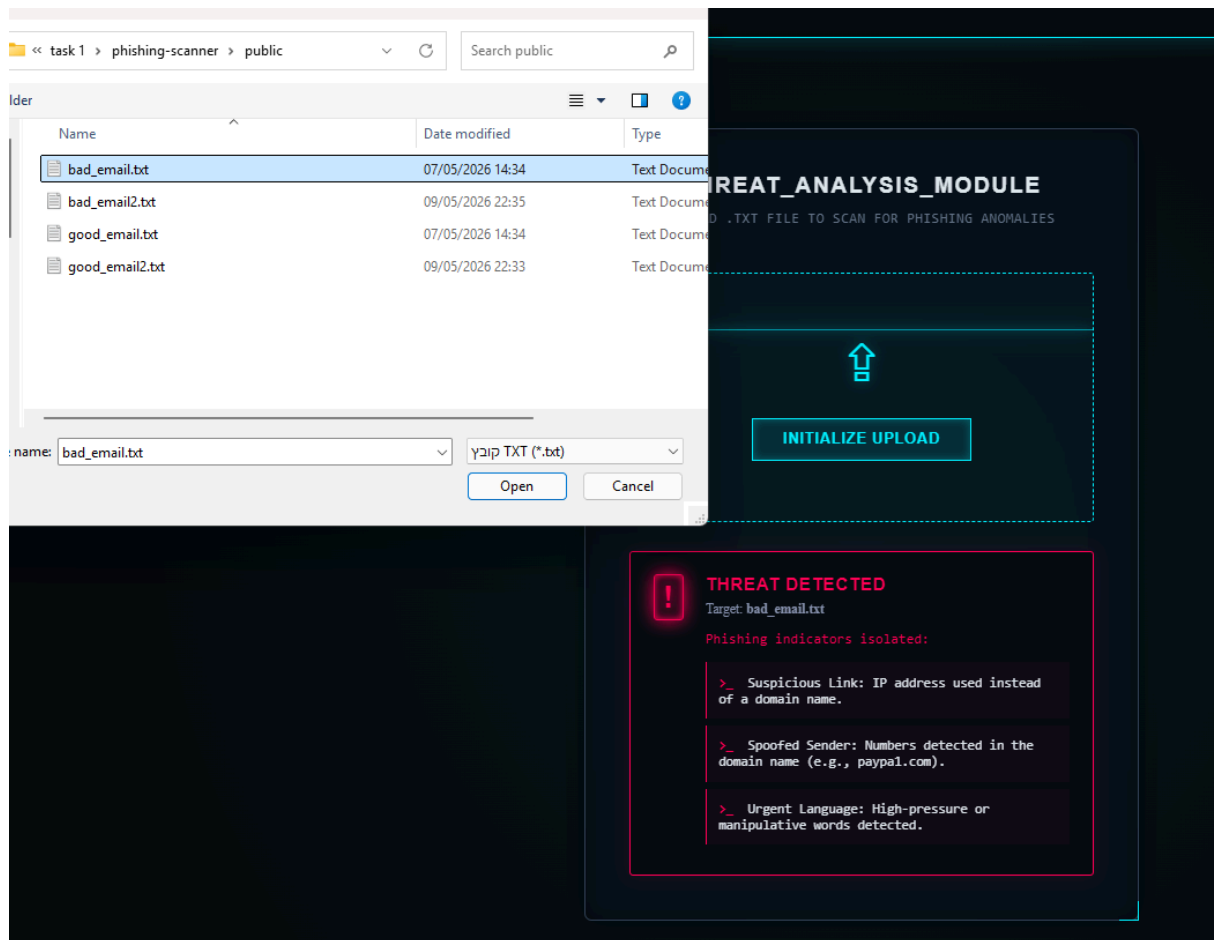
Tech Stack: Angular, TypeScript, SCSS, RxJS.

Highlights:

- Engineered a regex-based detection engine to identify spoofed domains, IP-based links, and urgent language.
- Built a modern, reactive Angular UI (Bonus Requirement) that processes `.txt` file uploads asynchronously.
- Included sample test files (`good_email.txt` and `bad_email.txt`) in the repository for easy reviewer testing.

Proof of Execution:





Task 2: Malware Analysis Sandbox

Objective: Create an isolated environment to execute and monitor simulated malware.

Tech Stack: Python, Docker, `inotifywait`, Tkinter (GUI).

Highlights:

- Utilized Docker to spin up a secure, isolated Ubuntu container containing an Angular web app as a realistic "honeypot."
- Monitored and logged File System modifications (`CREATE`, `DELETE`, `MODIFY`), Network Activity, and Process Activity (`ps aux`).
- Developed a complete Python Tkinter UI (Bonus Requirement) to control the sandbox lifecycle (Start, Run Malware, Generate Report, Stop/Clean) with a single click.

Proof of Execution:

Containers [Give feedback](#)

Container CPU usage ⓘ
0.01% / 800% (8 cores available)

Container memory usage ⓘ SI
1.94MB / 3.62GB

  Only show running containers

☐	Name	Image	Status	CPU (%)	Port(s)	Last started	Ac
☐	 Nexus_Malw	malware-sandbox-in	Running	0.01%		38 seconds ago	
☐	95b9a8383a0f						

 Malware Analysis Sandbox

Malware Sandbox Controller

1. Start

2. Run Malware

3. Generate Report

4. Stop & Clean

```
Starting Sandbox...
Sandbox running: 95b9a8383a0f
```

Malware Analysis Sandbox

Malware Sandbox Controller

1. Start

2. Run Malware

3. Generate Report

4. Stop & Clean

Executing malware...
Malware executed successfully

Generating report...
=== MALWARE ANALYSIS REPORT ===

[FILE SYSTEM MODIFICATIONS]

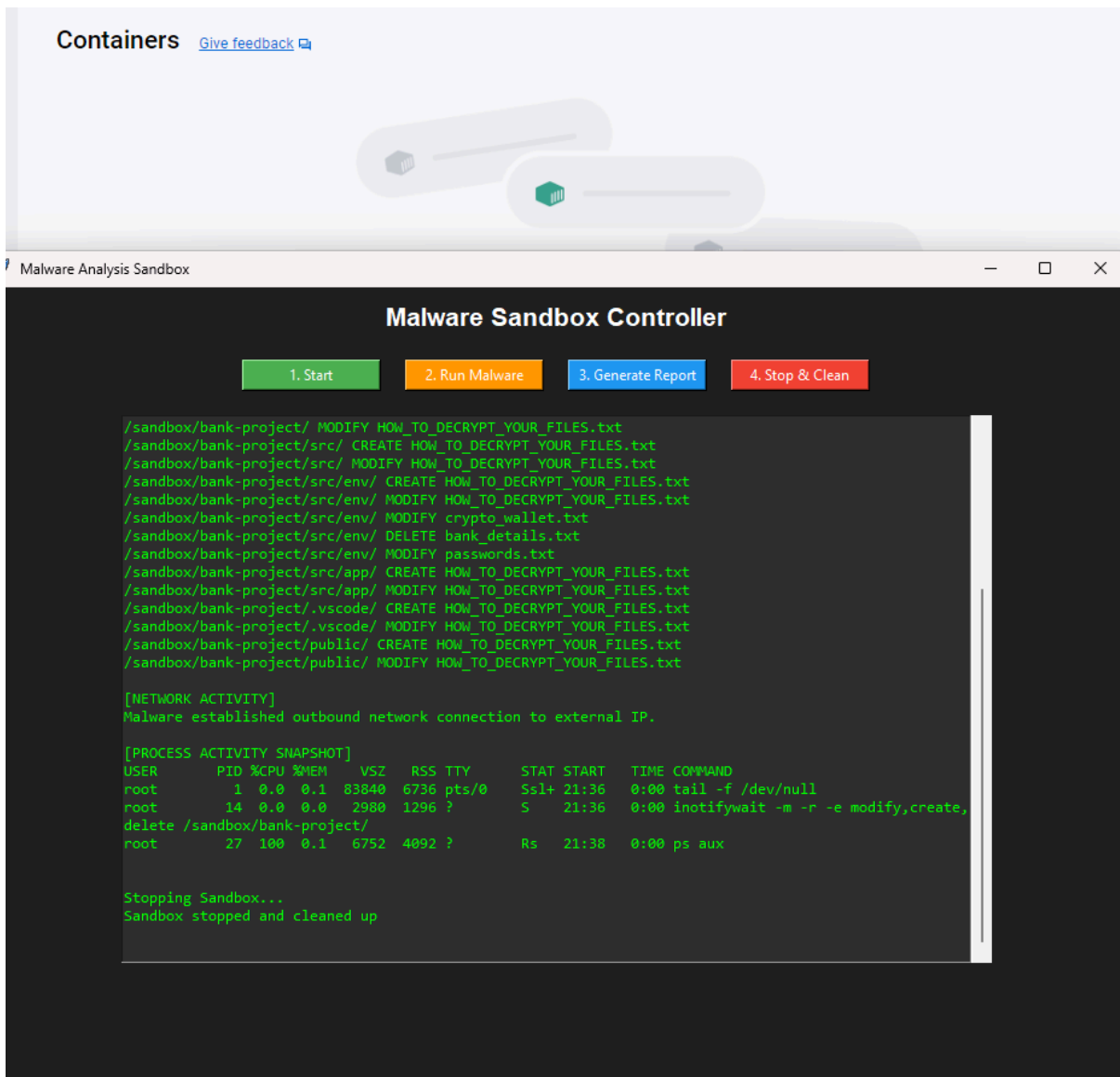
Setting up watches. Beware: since -r was given, this may take a while!
Watches established.
/sandbox/bank-project/ CREATE HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/ MODIFY HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/src/ CREATE HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/src/ MODIFY HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/src/env/ CREATE HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/src/env/ MODIFY HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/src/env/ MODIFY crypto_wallet.txt
/sandbox/bank-project/src/env/ DELETE bank_details.txt
/sandbox/bank-project/src/env/ MODIFY passwords.txt
/sandbox/bank-project/src/app/ CREATE HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/src/app/ MODIFY HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/.vscode/ CREATE HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/.vscode/ MODIFY HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/public/ CREATE HOW_TO_DECRYPT_YOUR_FILES.txt
/sandbox/bank-project/public/ MODIFY HOW_TO_DECRYPT_YOUR_FILES.txt

[NETWORK ACTIVITY]

Malware established outbound network connection to external IP.

[PROCESS ACTIVITY SNAPSHOT]

USER	PTD	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
------	-----	------	------	-----	-----	-----	------	-------	------	---------



Task 3: SQL Injection Simulation

Objective: Demonstrate a vulnerable login system, execute a SQL bypass, and provide a secure mitigation.

Tech Stack: Node.js, Express, SQLite, Angular.

Highlights:

- Built a side-by-side UI (Bonus Requirement) demonstrating a "Legacy" vulnerable system and a "Secure" modern system.
- Demonstrated a successful authentication bypass using the payload ' OR '1'='1 by exploiting direct string concatenation in the backend.

- Mitigated the vulnerability in the secure endpoint using parameterized queries, effectively neutralizing the payload.

Proof of Execution:

The image shows two side-by-side login forms. The left form is for the 'Legacy System v1.0' and the right is for the 'Secure System v2.0'. Both forms have 'USERNAME' and 'PASSWORD' fields and a 'LOGIN' button. In the Legacy System, the username field contains the payload `' OR '1'='1' --` and the password field contains `Enter password`. The login is successful, displaying a green message: 'Login Successful (Vulnerable)!'. In the Secure System, the username field contains the same payload `' OR '1'='1' --` and the password field contains `Enter password`. The login fails, displaying a red message: 'Invalid Username or Password'.

the second image is me putting the password:

`' OR '1'='1' --`

The image shows two side-by-side login forms. The left form is for the 'Legacy System v1.0' and the right is for the 'Secure System v2.0'. Both forms have 'USERNAME' and 'PASSWORD' fields and a 'LOGIN' button. In the Legacy System, the username field contains `nothing` and the password field contains `.....`. The login is successful, displaying a green message: 'Login Successful (Vulnerable)!'. In the Secure System, the username field contains `admin` and the password field contains `.....`. The login fails, displaying a red message: 'Invalid Username or Password'.

same result the username hack to put: `' OR '1'='1' --`

and password hack is: `' OR '1'='1'`